

Multimodal Obstacle Detection and Adaptive Neural Control for Autonomous Drones

T. Phetpoon[†], V. Jaiton[†], K. Rothomphiwat[†], M. Manawakul[†], P. Chirathanyanon, P. Ritmetee, P. Manoonpong^{*}

Abstract—Achieving reliable navigation for autonomous drones in complex environments remains a significant challenge, particularly in low-light conditions. To address this, we propose an integrated multimodal obstacle detection and adaptive neural control system with online learning to enable drones to navigate autonomously both during the day and at night. The proposed multimodal obstacle detection system integrates two ranging LiDAR sensors and a depth camera with sensory processing techniques, including the iKD-Tree interested area search algorithm, sensor fusion, and neuro-obstacle directional feature extraction. This ensures robust obstacle detection across various conditions without requiring sensor reconfiguration. The adaptive neural control system applies Hebbian correlation-based learning and synaptic scaling plasticity principles to continuously update the control weights, allowing the drone to dynamically adapt its speed and maneuver around obstacles in real time. We evaluate the system's performance in both simulation and real-world environments, demonstrating its effectiveness under diverse lighting conditions and obstacle types.

Index Terms—Drones, Multimodal sensory system, Adaptive neural control, Obstacle avoidance, Speed adaptation

I. INTRODUCTION

Autonomous drones have seen significant adoption across various industries, including inspections, agriculture, and delivery services [1]. Many drone applications require autonomous operation with the ability to avoid obstacles. Furthermore, during flight missions, drones frequently encounter diverse and unexpected obstacles, requiring adaptive strategies for safe and efficient navigation. Therefore, developing systems that enable drones to autonomously adjust their flying speed and maneuver around obstacles is essential for enhancing their operational reliability and safety. Recent developments in autonomous drone navigation have focused on enhancing obstacle detection and avoidance capabilities.

Various navigation approaches have been proposed, including SLAM-based, graph-based, optimization-based, reactive-based, and learning-based methods. The SLAM-based methods in [2] enable autonomous flights and real-time semantic



Fig. 1: An illustration of an autonomous drone's ability to adapt speed and avoid obstacles in both day and night, powered by the proposed adaptive neural control system with multimodal obstacle detection. A video demonstration is available at <https://youtu.be/agZfvFkkVxY>.

mapping by utilizing 3D LiDAR to detect the environment. They employ a multi-level planning and mapping framework to navigate the drone efficiently while minimizing odometry drift and maintaining planning accuracy. Graph-based methods [3], Rapidly-exploring Random Tree (RRT) [4], and the Probabilistic Roadmaps (PRM) [5] algorithm model the environments as a graph, where nodes represent locations or states in obstacle-free space, and edges indicate connections or transitions between them. These approaches facilitate navigation by traversing the graph and making decisions based on optimization techniques utilizing real-time data collected during navigation. Optimization-based methods [6], [7] minimize the cost function to enable drones to dynamically adjust their motion and decision-making while navigating real-world environments. These methods optimize trajectories, speed, and obstacle avoidance in real time by processing sensor data and adapting to changing conditions. Another optimization-based approach, model predictive control (MPC) [8], computes optimal trajectories over a defined time horizon while incorporating mathematical drone models and system constraints. These three methods (SLAM-based, graph-based, and optimization-based) navigate by optimizing decisions based on real-time data but rely heavily on accurate environmental perception. Therefore, advanced detection systems like 3D LiDAR and depth cameras are essential. Additionally, they require significant computational resources, especially in large or complex environments.

[†]These authors contributed equally to this work.

T. Phetpoon, V. Jaiton, K. Rothomphiwat, M. Manawakul, and P. Manoonpong are with the Vidyasirimedhi Institute of Science and Technology (VISTEC), Rayong, Thailand

P. Chirathanyanon and P. Ritmetee are with the AI and Robotics Ventures (ARV), Thailand

^{*}Corresponding author: poramate.m@vistec.ac.th

This work was supported by AI and Robotics Ventures (ARV). We thank the ARV team for participating in the technical discussions.

Supplementary Material: https://vistec-ist-robotics.github.io/IROS25_MODANC/.

The reactive control methods, like potential field [9], model the drone movement in an artificial field where attractive forces pull it toward the goal while repulsive forces push it away from obstacles. This approach enables smooth, dynamic motion and is computationally efficient, making it suitable for environments with unpredictable obstacles. However, it has limitations, such as the local minima problem, where forces cancel out and trap the drone and oscillatory behavior near obstacles. Additionally, it lacks global path planning, as it reacts locally without optimizing the overall path.

Learning-based methods use machine learning to enable drones to autonomously navigate unknown environments. Deep reinforcement learning (DRL) [10], [11] is integrated with deep neural networks to enhance drone navigation in complex environments. Through trial and error, drones refine navigation policies by interacting with the environment, receiving rewards, and adjusting actions to maximize long-term gains. While effective, DRL requires comprehensive training datasets, simulated training environments, and substantial computational resources for optimal learning.

These navigation approaches—including SLAM-based, graph-based, optimization-based, reactive-based, and learning-based methods—are typically validated under daytime conditions, thereby limiting their applicability in low-light or nighttime scenarios.

Based on the aforementioned challenges, we propose an integrated multimodal obstacle detection and adaptive neural control system with online learning to enable the drone to navigate autonomously both during the day and at night and deal with various obstacle sizes. The proposed multimodal system uses a LiDAR sensor and a depth camera for obstacle detection under varying lighting conditions and can be extended with additional modalities to further enhance perception. Efficient and robust sensory processing techniques, such as the iKD-Tree interested area search algorithm, sensor fusion, and neuro-obstacle directional feature, are applied to achieve these detection tasks. The adaptive neural control integrates speed adaptation and obstacle avoidance modules by applying Hebbian correlation-based learning and synaptic scaling plasticity principles to continuously adapt the control weights in the neural control system. This approach enables the drone to navigate autonomously under both conditions. We evaluate the system's performance and demonstrate the drone's navigation behaviors in both simulated (Gazebo) and real-world environments. Taken together, this study makes the following key contributions:

- A multimodal obstacle detection combining two ranging LiDAR sensors, a depth camera, and an efficient, robust sensory processing technique to detect obstacles in both daytime and nighttime conditions.
- An adaptive neural control system with online learning based on Hebbian correlation-based learning and synaptic scaling plasticity to adaptively navigate the drone through both daytime and nighttime conditions in obstacle-filled environments.
- A system performance evaluation and behavior demonstration in simulated and real-world environments.

II. MATERIALS AND METHODS

In this section, we describe the multimodal obstacle detection and adaptive neural control with online learning. First, we provide an overview of the system. Next, we explain the multimodal obstacle detection and adaptive neural control. Finally, we describe the post-processing module.

A. System Overview

The proposed system comprises six main components, progressing from sensory inputs to drone control (see Fig. 2): (i) Sensory Inputs: Two ranging LiDARs and a depth camera provide raw environmental data. (ii) Sensory Pre-processing: Raw point cloud data from the field of view (FOV) of a depth camera are filtered to a region of interest and converted into laser scan data. (iii) Multimodal Sensory Integration: This part extracts nearest distance and directional obstacle features from all sensory inputs. (iv) Adaptive Neural Control: This module integrates obstacle avoidance and speed adaptation for safe flight navigation. (v) Post-Processing Module: This module translates neural outputs into control commands for the flight controller. (vi) Flight Controller: The low-level flight controller finally generates control commands for drone operation.

B. Sensory Pre-processing System

The sensory pre-processing system processes data from a depth camera and LiDAR sensors. It consists of two main components: the nearest-point search and the data segmentation module. The nearest-point search efficiently identifies the closest obstacle within the point cloud, while the data segmentation module organizes and structures the data for further analysis.

1) Nearest-point Search via Incremental K-D Tree (iKD-Tree)

Efficiently identifying the nearest obstacle within a dense point cloud is crucial for real-time perception. The depth camera captures 896×512 points within a 90° (H) $\times$ 60° (V) field of view, resulting in a large dataset requiring a fast nearest-point search.

To address this, we employ an iKD-Tree [12], an efficient data structure that enables dynamic point cloud processing. Specifically, it allows us to rapidly locate the closest point within an interested area around the drone that may potentially hit an obstacle while continuously updating with new data (see Fig. 3). The iKD-Tree supports efficient insertion, deletion, and down-sampling, significantly reducing computational overhead while maintaining search accuracy. Additionally, sensor fusion with LiDAR data further enhances the reliability of proximity estimation.

2) Laser Scan Segmentation Process

The segmentation process begins with point clouds from the depth camera (see Fig. 2). Based on the raw point cloud data of 896 (width) $\times$ 512 (height) points, we transform the input from point cloud format to laser scan format (896 (W) $\times$ 1 (H)) using the point cloud-to-laser scan library. The laser scan data is then divided into ten segments (10 (W) $\times$ 1 (H)), each covering 9 degrees. The ten mapped

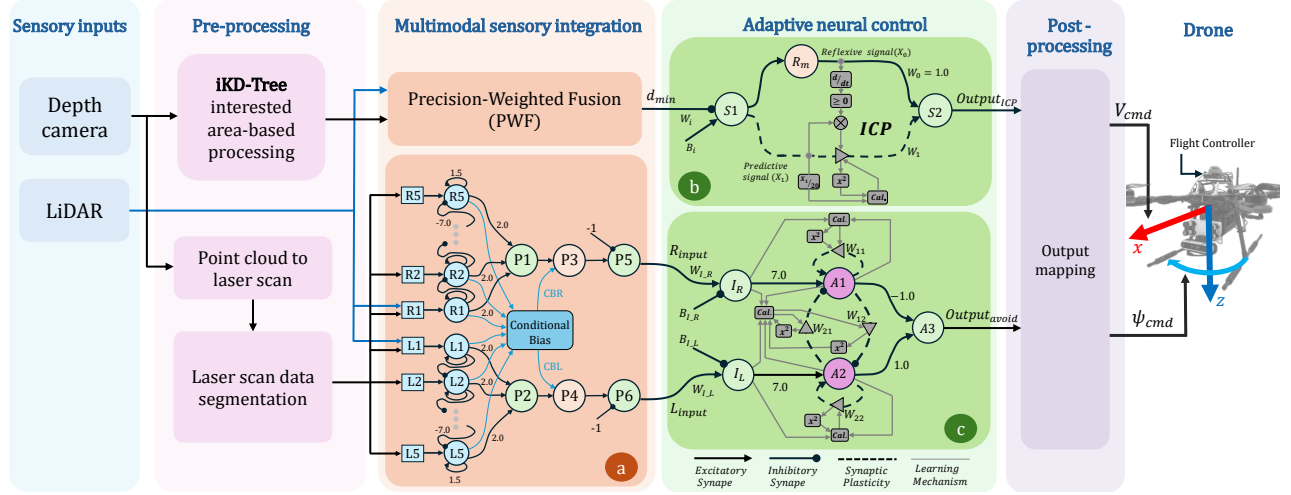


Fig. 2: System overview of sensory inputs (we use two ranging LiDAR sensors and a depth camera), a pre-processing module, multimodal sensory integration including PWF and obstacle directional feature extraction (a), adaptive neural control with speed adaptation (b) and obstacle avoidance (c) algorithms, a post-processing module, and a flight controller mounted on a drone.

inputs are labeled R5, R4, R3, R2, R1, L1, L2, L3, L4, and L5. These ten segmented data points serve as inputs for further computation in the obstacle directional feature extraction module, as described later.

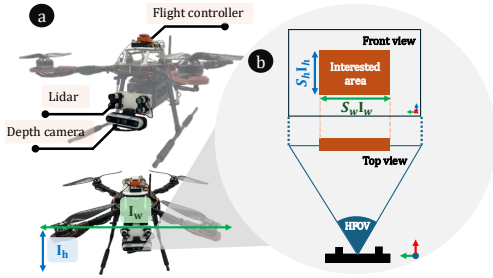


Fig. 3: (a) Drone components. (b) Interested area. It is represented as $S_h I_h \times S_w I_w$. The area highlights the most critical zone that the drone may potentially hit. I_h and I_w are defined by the drone's dimensions. S_h and S_w are defined by the safety factors of each dimension.

C. Multimodal Sensory Integration

1) Precision-Weighted Fusion (PWF)

In this sensor fusion system, we employ a PWF method to combine the closest distance estimates from the camera and lidar sensors, leveraging their uncertainties for an optimal estimate. This approach, based on Kalman filtering [13] and Bayesian inference, determines each sensor's contribution based on its variance.

Let $X \sim \mathcal{N}(\mu_X, \sigma_X^2)$ and $Y \sim \mathcal{N}(\mu_Y, \sigma_Y^2)$, where X and Y are random variables representing the distance estimates from the first and second sensors, respectively. The parameters μ_X , σ_X^2 and μ_Y , σ_Y^2 denote their respective means and variances.

The fused random variable Z is calculated as a weighted sum of X and Y , with weights inversely proportional to their variances. The fused mean and variance are:

$$\mu_Z = w_X \mu_X + w_Y \mu_Y, \quad \sigma_Z^2 = \frac{\sigma_X^2 \sigma_Y^2}{\sigma_X^2 + \sigma_Y^2}, \quad (1)$$

where the weights are:

$$w_X = \frac{\sigma_Y^2}{\sigma_X^2 + \sigma_Y^2}, \quad w_Y = \frac{\sigma_X^2}{\sigma_X^2 + \sigma_Y^2}, \quad (2)$$

Thus, the fused Gaussian distribution is $Z \sim \mathcal{N}(\mu_Z, \sigma_Z^2)$. The final output of this pre-processing module is then used as input to the speed adaptation algorithm in the adaptive neural control.

2) Obstacle Directional Feature Extraction

The obstacle directional feature extraction receives ten segmented inputs from the segmentation process (see Fig.2). Each input is linearly mapped from [min range, max range] to the range [-10, 10] and then fed into its corresponding recurrent neuron, from R5 to L5, respectively. Additionally, neurons R1 and L1 receive inputs from the right and left LiDAR sensors. Apart from receiving input from the processed point clouds and LiDAR sensors, each neuron also receives recurrent input from its own output, with a weight of 1.5. Moreover, the output of R1 is applied as input to R2 with a weight of -7.0 to inhibit R2's activation. This pattern is repeated, with the output of R2 applied to R3, and so on for the remaining neurons, including L1 through L5. The outputs of R1 through R5 are transferred as inputs through the weights of 2.0 to neuron P1, a linear summation neuron, while the outputs of L1 through L5 are transferred as inputs to neuron P2 through the same weights value. Next, the outputs of P1 and P2 are applied as inputs to P3 and P4, respectively, along with conditional bias inputs (CBR and CBL). The conditional bias performs a summation comparison of five right and five left neurons when both sides are activated simultaneously. If the summed activity of the right neurons exceeds that of the left neurons, the outputs CBR and CBL are set to 0.0 and -10.0, respectively, indicating a stronger obstacle presence on the right. Conversely, if the left neurons have a higher summed activity, the outputs are reversed. Finally, the outputs of P3 and P4 are transferred to neurons P5 and P6, respectively, combined with bias values

of -1.0 . This neuro-obstacle directional feature computation is divided into two sides: the right ($P1$, $P3$, and $P5$) and left ($P2$, $P4$, and $P6$), with the outputs of $P5$ and $P6$ representing the directional feature of the detected obstacle. All neurons in this computation are modeled as discrete-time non-spiking neurons, with their output governed by Eq. 3. These outputs are then used as inputs for the obstacle avoidance algorithm in the adaptive neural control, as explained in the next section.

$$O_i(t) = f \left(\sum_{j=1}^n W_{ij}(t) \cdot O_j(t-1) + B_i \right) \quad i = 1, \dots, n, \quad (3)$$

where n denotes the number of neurons, and B_i is the internal bias term to neuron i . W_{ij} represents the synaptic weight of the connection from neuron j to neuron i . $f()$ denotes the activation function, which includes a sigmoid activation function (for the blue neurons in Fig. 2(a)), a linear activation function (for the green neurons in Fig. 2(a)), and a modified ReLU activation function (for the orange neurons in Fig. 2(a)).

D. Adaptive Neural Control with Online Learning

The adaptive neural control module used here comprises two algorithms: (i) adaptive speed control and (ii) adaptive obstacle avoidance control. Both algorithms receive separate inputs from the pre-processing module. Online learning is applied to enable the system to learn the control parameters (synaptic weights) while performing missions, employing the synaptic plasticity principle. All neurons in this system are modeled as discrete-time, non-spiking neurons, with their outputs governed by Eq. 3.

1) Adaptive Speed Control

The adaptive speed control (see Fig. 2(b), green box) is an enhanced version of [14], originally relying on an unsupervised learning mechanism based on an input correlation-based (ICO) learning principle. ICO learning follows an incremental learning concept, limiting the system's ability for lifelong learning. Here, we incorporate ICO learning with a synaptic scaling plasticity mechanism (SSP) called input correlation plasticity (ICP) to enable lifelong learning, allowing the system to perform online learning during missions. The main computation of the adaptive speed control is based on a three-neuron network ($S1$, R_m , and $S2$). $S1$ receives a constant (bias) input, B_i , and sensory input from the PWF (see 2, orange box) through the weight W_i . The output of $S1$ is then split into predictive and reflexive signals, each representing a distinct obstacle detection range for the ICP learning mechanism. The predictive signal (X_1) is responsible for long-range obstacle detection, enabling early anticipation, while the reflexive signal (X_0) handles short-range detection for immediate responses. To achieve this, $S1$ functions as a linear neuron with an input mapping weight (W_i) and a bias input (B_i), aligning with the detection ranges and drone size. The output of $S1$ is directly used as the predictive signal (X_1). The reflexive signal (X_0), on the other hand, is obtained by passing the $S1$ output through a modified ReLU neuron (R_m), which outputs 0.0 if the

input is less than 1.0, and the input value itself if it is equal to or greater than 1.0. These two signals are then fed into the neuron $S2$ via their respective synaptic weights to produce an adaptive output ($S2$ output) for speed adaptation control. In this setup, the reflexive signal is operated through a fixed-speed control gain (reflexive weight, W_0), set to 1.0, allowing the drone to reactively reduce speed when a short-range obstacle is detected. Meanwhile, the predictive signal is processed through an adaptive speed control gain (predictive weight, W_1), learned via the ICP mechanism as a lifelong online learning process, enabling proactive speed reduction for distant obstacles in uncertain environments during missions. The ICP learning mechanism is described as follows:

$$\Delta W_1 = \underbrace{\mu_s \frac{dX_0(t)}{dt} X_1(t)}_{ICO} - \underbrace{\gamma_s (k_s + P(t))(W_1(t))^2}_{SSP} \quad (4)$$

where ΔW_1 denotes the change in the predictive weight value at the next time step. μ_s is the ICP learning rate, which defines how fast the system learns and is set to 0.3. $X_0(t)$ is a reflexive signal where only the positive values of its derivative are used. $X_1(t)$ is a predictive signal. γ_s is the ICP forgetting rate, set to 0.0001. k_s is an ICP offset that enables a constant reduction of the weights when the system is inactive; here, it is set to 0.01. $P(t)$ is a plasticity factor, as described in Eq. 5. $W_1(t)$ represents the predictive weight at the current time step. It should be noted that all variables defined here are empirically determined.

$$P(t) = \begin{cases} 0.05 \cdot X_1(t) & X_1(t) > 0, \\ 0 & X_1(t) \leq 0, \end{cases} \quad (5)$$

where $X_1(t)$ is the predictive signal.

2) Adaptive Obstacle Avoidance Control

The adaptive obstacle avoidance control (see Fig. 2(c)) processes two input signals derived from the neuro-obstacle directional feature extraction system. The core of this algorithm is two interconnected recurrent neurons ($A1$ and $A2$), as described in Eq. 3, each using the tanh activation function and receiving input weighted at 7.0. Self-connection weights (W_{11} and W_{22}) introduce a hysteresis effect in the neural activations, allowing the drone to continue turning even after obstacles are no longer detected. In addition to these self-connections, the neurons are also interconnected by two inhibitory synapses (W_{12} and W_{21}). This configuration ensures that when one neuron is activated, it inhibits and delays the activation of the other. This mechanism supports stable, uninterrupted turning behavior, even in scenarios where obstacles appear symmetrically at the center of the drone's field of view.

For this implementation, we allow all weights (W_{11} , W_{22} , W_{12} , and W_{21}) to learn online during the missions using the learning mechanism and parameters derived from [15]. The recurrent excitatory weights ($W_{11,22}$) are governed as follows:

$$\Delta W_{ii} = \mu_e v_i(t-1) v_i(t) R_i(t) + \gamma(k - v_i(t)) W_{ii}(t)^2, \quad (6)$$

$$W_{ii}(t+1) = W_{ii}(t) + \Delta W_{ii}, \quad (7)$$

where ΔW_{ii} is the weight value change in a time step; μ_e is a learning rate for the excitatory synapses, set to 0.01; γ is a forgetting rate, set to 0.0003; k is an offset that enables a constant reduction of the weights when no obstacle is detected, set to -0.01; and i represents 1, and 2. $R_i(t)$ and $v_i(t)$ are the reflexive signal and the positive mapped output for stability constraint, respectively, which are described below:

$$R_i(t) = \begin{cases} 1 & I_i(t) > -0.5, \quad i = 1, 2, \\ 0 & I_i(t) \leq -0.5, \quad i = 1, 2, \end{cases} \quad (8)$$

$$v_i(t) = 0.5o_i(t) + 0.5, \quad i = 1, 2. \quad (9)$$

The inhibitory weights (W_{12} and W_{21}) are updated in a similar manner, with equal adjustments applied to maintain symmetry. This is achieved by computing the average of their updated values and assigning it to both W_{12} and W_{21} , as follows:

$$\Delta W_i = \mu v_i(t-1)v_i(t)R_i(t) + \gamma(k - v_i(t))W_i(t)^2, \quad (10)$$

$$\Delta W_j = \mu v_j(t-1)v_j(t)R_j(t) + \gamma(k - v_j(t))W_j(t)^2, \quad (11)$$

$$W_i(t+1) = W_i(t) - \Delta W_i, \quad (12)$$

$$W_j(t+1) = W_j(t) - \Delta W_j, \quad (13)$$

$$W_{ij}(t+1), W_{ji}(t+1) = 0.5W_i(t) + 0.5W_j(t), \quad (14)$$

where ΔW_i and ΔW_j are temporary variables used to compute the changes in the inhibitory weights; W_i and W_j are temporary variables used to calculate the average inhibitory weights; μ is the learning rate for inhibitory synapses, set to 0.015; γ is the forgetting rate, and k is the offset—both set to the same values as previously defined; and i and j represent neurons 1 and 2, respectively.

The outputs of neurons A1 and A2 are processed by a linear neuron (A3) with synaptic weights of -1.0 and 1.0, respectively. The output of A3 determines the drone's turning direction: a positive value causes the drone to turn right when an obstacle is detected on the left, while a negative value causes it to turn left when an obstacle is detected on the right. Importantly, this turning behavior is adaptively modulated through the online learning of the synaptic weights W_{11} , W_{22} , W_{12} , and W_{21} . These weights are continuously updated in real-time based on inputs from a depth camera and LiDARs as well as ongoing dynamical interactions with the environment.

E. Post-processing Module

The outputs from the adaptive speed and obstacle avoidance controls are finally mapped to low-level control commands through the post-processing module (see the Supplementary Material for details). The adaptive speed output is mapped to a flight speed command using a state machine-based mapping function. Simultaneously, the obstacle avoidance output is combined with the directional goal. The resulting directional control command guides the drone toward its goal while enabling it to avoid obstacles along the flight path.

III. EXPERIMENTS AND RESULTS

Three main experiments were conducted to evaluate and demonstrate the performance of our proposed system. First, we assessed the effectiveness of multimodal sensory detection by evaluating the system's sensory inputs and integration. Second, we tested and validated the system's performance in a simulated environment. Finally, we tested the system in real-world conditions to verify its practical applicability.

A. Evaluation of Multimodal Obstacle Detection

We evaluated the robustness of a sensor fusion approach that integrates measurement distances from the left and right LiDAR sensors ($\delta_{lidar_{left}}$, $\delta_{lidar_{right}}$) with the nearest distance estimate from a depth camera with its infrared (IR) functionality disabled (δ_{camera}) to determine the closest obstacle distance ($\delta_{estimate}$ or d_{min}). The experiment assessed the system's performance under varying lighting conditions and obstacle sizes. Specifically, tests were conducted under two lighting conditions: low-light (0.76 lux) and well-lighted (785 lux). To analyze detection performance at different scales, obstacles of varying sizes were introduced, including a small-sized object (i.e., a pole with a 30 cm diameter), a medium-sized object (i.e., a cart with dimensions of 69.5 x 110 cm), and a large-sized object (i.e., a flat board with dimensions of 152.5 x 159 cm). Performance was evaluated by measuring the absolute error between the estimated distance from each measurement and the reference distances, ranging from 0.5 to 7.0 meters. Each measurement point collected a sample size of 100 to determine the mean and variance of the measurement. The results were categorized according to lighting conditions and obstacle sizes, as shown in Fig. 4.

The results indicated that in low-light conditions, the depth camera struggled, leading to higher errors, whereas LiDAR maintained accuracy. Under well-lighted conditions, depth estimation improved significantly. Larger obstacles were detected more reliably. Sensor fusion improved overall performance by utilizing LiDAR for reliable detection in darkness and depth data in well-lighted conditions for obstacles of varying sizes, ensuring robust obstacle detection in many cases. However, limitations persisted in low-light environments, where the depth camera performed poorly, and the LiDAR sensors had difficulty detecting small obstacles.

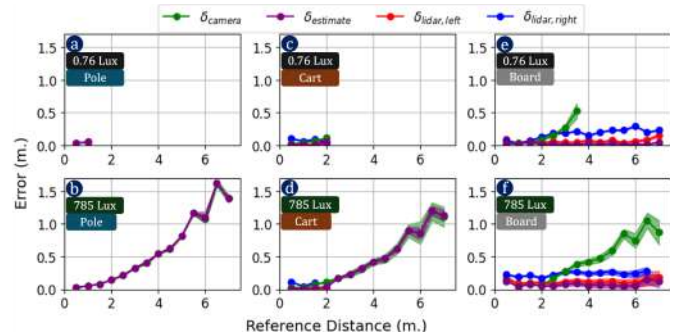


Fig. 4: Measurement errors of different sensors across reference distances (0.3–7.0 m) under varying lighting conditions.

B. Simulated Experiments

In this experiment, we used an IRIS drone in a Gazebo classic simulation with PX4 SITL as the experimental platform. The drone has dimensions of $0.55 \times 0.1 \text{ m}^2$ ($W \times H$) and a total weight of 1.5 kg. Consequently, we set the area of interest for the iKD Tree processing to $1.5 \times 0.3 \text{ m}^2$ ($W \times H$), approximately three times the dimensions of the drone. Additionally, the minimum and maximum ranges for mapping the segmented inputs were empirically set to 1.0 m. and 4.0 m., respectively. These values also corresponded to the short-range (reflexive range) and long-range (predictive range) detection in the adaptive speed control system. The computational setup consists of a laptop equipped with an Intel Core i7-8750H CPU and an NVIDIA GTX 1050 Ti GPU. Based on this setup, we conducted two sub-experiments to evaluate our system's performance: (i) comparison with a baseline method and (ii) evaluation against other state-of-the-art methods.

1) Online Learning Baseline Comparison

In this online learning comparison, we analyzed how different learning mechanisms influenced the system's performance by comparing ICO learning (the baseline method, see the first term of Eq. 4) with ICP learning (our enhanced method, see Eq. 4). The drone autonomously navigated at a speed of 2.0 m/s from waypoint (1) to (2), and upon reaching (2), the goal shifted to waypoint (3), continuing this process until it reached waypoint (5). The evaluation was conducted in two complex environments: one with high pole density and another resembling a forest environment with high tree density (see Figs. 5(a) and (b)). Each experiment was repeated ten times in both environments for each learning mechanism.

The results in Fig. 5(c) demonstrate that using ICP learning as an online learning mechanism enabled the system to autonomously navigate the drone through both pole and forest environments with a 100% success rate, completing all missions without colliding with any obstacles. In comparison, ICO learning achieved success rates of 80% in the pole environment and 40% in the forest environment. This is due to the incremental learning concept of ICO learning, where the predictive weight (W_1) can only increase. As a result, when online learning is enabled to navigate the drone through complex environments, the learned predictive weight (W_1) reaches its optimal value. However, the weight continues to be updated when the drone encounters an obstacle—whether by avoiding one and facing another or by encountering a suddenly appearing obstacle. This leads to a non-optimal weight value, causing the drone to get stuck, failing to complete the mission. However, by implementing ICP learning, the weight can be "learned" online (see the first term in Eq. 4 to handle complex environments when obstacles are detected, and it can also be "forgotten" (see the second term in Eq. 4) when no obstacles are present. As a result, the weight exhibits plasticity, allowing it to adjust around the optimal value. The comparison of learning behavior between both methods is presented in Fig. 5(d).

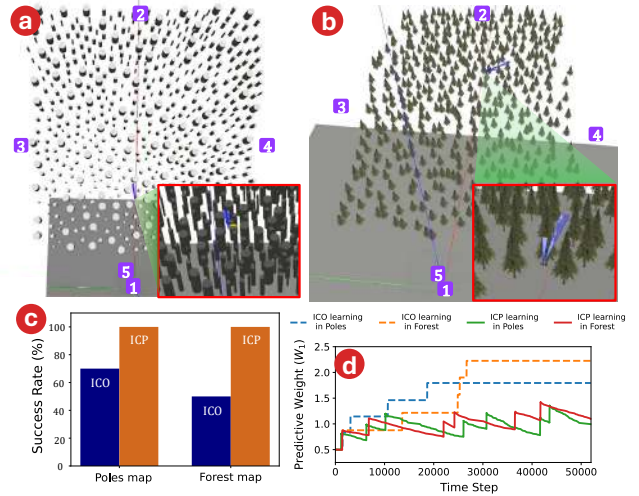


Fig. 5: Testing environments for online learning evaluation: (a) A high pole density environment and (b) a forest environment. (c) Success rate comparison. (d) Predictive learning weights in different conditions. Video demonstrations, both simulation and real-world, are available in the Supplementary Material.

2) System Performance Comparison

To compare the performance of the proposed system with other state-of-the-art methods, we conducted two evaluation tests. The first assessed computational latency, while the second compared success rates in three distinct environments: long walls, poles, and a dynamic pole (suddenly appearing).

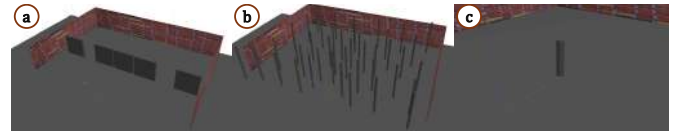


Fig. 6: The different environment for comparison all planning methods. (a) Long wall. (b) Poles. (c) Dynamic pole.

Computational Latency: This comparison aims to evaluate the efficiency of our approach in handling real-time computations while ensuring reliable navigation performance. To achieve this, we measured the processing time of our system and compared it with [7] and [9] optimization-based and reactive-based control methods, respectively. Additionally, we analyzed the latency of each component separately, focusing on (1) perception processing and (2) planning and control. The results in Table I show that our method had a total processing time of 42.15 ms., compared to 17.59 ms. for [7] and 259.48 ms. for [9]. Interestingly, when analyzing the components separately, we found that our method primarily spends time on perception processing, which takes 41.72 ms.—significantly higher than both [7] and [9], which require 7.64 ms. and 13.28 ms., respectively. However, in the planning and control stage, our method proved to be highly efficient, requiring only 0.43 ms.—considerably less than [7] and [9], which take 9.95 ms. and 246.21 ms., respectively. Notably, perception processing for ICP included iKD-Tree and PWF, while perception processing for avoidance consisted of point cloud-to-laser scan conversion, laser scan data segmentation, and obstacle directional feature extraction.

TABLE I: Comparison of planning and control latency

Method	Components	Proc. Time (ms)	Total Proc. Time (ms)
Falco [7]	Perception process	7.64	17.59
	Control process	9.95	
PAPF [9]	Perception process	13.28	259.48
	Control process	246.21	
ICP	Perception process (Speed)	41.72*	42.15
	Control process (Speed)	0.27	
	Perception process (Avoidance)	14.08*	
	Control process (Avoidance)	0.16	

* Highlighted processes run in parallel, so the process with the maximum time was selected as the total processing time.

Note that for the ICO method, the result is identical to the result of ICP method.

Success Rate: To evaluate system performance, we designed three different simulated environments: large obstacles (long walls), multiple obstacles (poles), and dynamic obstacle (a dynamic or suddenly appearing pole). The system was tested through repeated missions in each environment, measuring the mission completion time and performing ten runs per scenario to evaluate the success rate. The results in Table II show that our method achieved a 100% success rate in all environments, while both [7] and [9] failed in the long-walled environment. Additionally, [9] achieved low success rates of 80% in the environment with multiple poles and 70% in the environment with a dynamic pole. Apart from these, both methods achieved a 100% success rate. In terms of mission completion time, our method completed missions approximately twice as fast as [9], but slower than [7].

TABLE II: Comparison of planning methods in different environments

Method	Environment	Avg. time (s)	Success (%)	Max Speed (m/s)
Falco [7]	Long wall	-	not success	2.0
	Poles	21.270	100	
	Dynamic pole	15.942	100	
PAPF [9]	Long wall	-	not success	1.0
	Poles	87.114	80	
	Dynamic pole	46.673	70	
ICO [14]	Long wall	48.614	100	2.0
	Poles	38.599	100	
	Dynamic pole	23.215	100	
ICP	Long wall	49.485	100	2.0
	Poles	40.191	100	
	Dynamic pole	23.939	100	

* A successful flight means the quadrotor reaches the goal without collisions.

C. Real-world Experiments

To evaluate the practical applicability of our approach, we deployed it on a quadrotor drone (see Fig. 3(a)) and conducted real-world experiments across various scenarios. The drone was equipped with a Tarot 650 frame, 14-inch propellers, and T-Motor MN4010/KV370 brushless motors. Key hardware components included a CubePilot (Cube Orange) flight controller with a built-in IMU running PX4 autopilot V1.14 firmware and an Nvidia Jetson Orin NX running JetPack 6.0 as the flight computer, which processed our algorithms in real time. The input sensors consisted of a RealSense D455 camera for depth imaging and two Garmin LiDAR Lite V3 units for LiDAR-based measurements. The drone had dimensions of $0.80 \times 0.30 \text{ m}^2$ (W $\times$ H) and a total weight of 3.2 kg. Based on these dimensions, we set the iKD-Tree processing area to $2.4 \times 1.5 \text{ m}^2$ (W $\times$ H), approximately three times the drone's width and five times its height. The height factor was increased from three to five to account for the altitude variations observed during real-world tests. For

the mapping of segmented input ranges, the minimum and maximum values were empirically set to 2.0 m and 7.0 m, respectively. These values corresponded to the short-range (reflexive range) and long-range (predictive range) detection used in the adaptive speed control system.

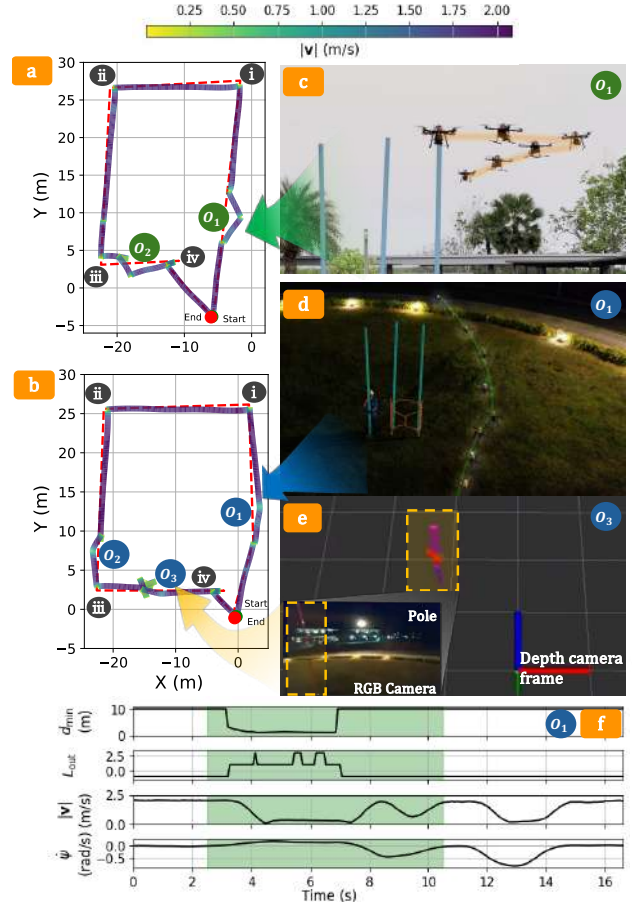


Fig. 7: Drone trajectory during the mission in both daytime (a) and nighttime (b). The predefined mission path is shown as a red dashed line, and the drone's flight path and speed are represented by a gradient line. O_i represents the i th obstacle that occludes the flying path. Waypoints (i-iv) represent the predefined waypoints. (c) shows the obstacle avoidance snapshots at O_1 during the day, (d) shows the obstacle avoidance snapshots at O_1 during the night, (e) The point cloud and RGB image of a detected dynamic obstacle during the nighttime mission, and (f) illustrates the drone's speed and angular velocity during obstacle avoidance, with the green area indicating when nearby obstacles were detected (as d_{min} decreases) and an obstacle feature was identified (as L_{out} rises), leading to an increase in yaw rate to adjust the heading. A video demonstration is available at https://youtu.be/Lsm5-47Ur_4.

The tests were conducted in outdoor environments under both daytime and nighttime conditions (see Figs. 1 and 7). The drone was tasked with the following predefined waypoints (i-iv) along a designated path, with the maximum speed set to 2.0 m/s. During the mission, static obstacles blocked the direct path between waypoints. Three poles, each with a diameter of 115 mm, were arranged in a straight line to simulate narrow gaps and small obstacles. The results shown in Fig. 7 demonstrate that the drone successfully follows the predefined waypoints and completes the same path under both daytime and nighttime conditions. To assess the system's robustness, dynamic obstacles were introduced

during the nighttime experiment. A pole (diameter: 3 cm; see Fig. 7(e)) was manipulated by a human to suddenly appear in the drone's perception system, approximately 2 m away. The drone successfully avoided the dynamic obstacle and completed the mission. Fig. 7(b) shows the drone's path at point O_3 as it encounters the dynamic obstacle.

IV. DISCUSSION AND CONCLUSION

In this study, we introduce a multimodal obstacle detection and adaptive neural control system with online learning, enabling autonomous drone navigation in both daytime and nighttime conditions. The system integrates two rangefinder LiDAR sensors and a depth camera to ensure reliable obstacle detection regardless of lighting conditions. To enhance efficiency and robustness, advanced sensory processing techniques—such as the iKD-Tree interested area search algorithm, precision-weighted fusion (PWF), and neuro-obstacle directional feature extraction—are employed to optimize obstacle detection. The adaptive neural control system combines speed adaptation and obstacle avoidance using unsupervised Hebbian correlation-based learning and synaptic scaling plasticity, referred to as input correlation plasticity (ICP), to continuously adjust neural weights. This online learning capability allows the drone to navigate autonomously without requiring prior training. The system enables the drone to explore the target area and dynamically optimize control parameters during the mission, ensuring efficient and adaptive navigation.

The performance and robustness of the proposed system were evaluated on both simulated and real drones by comparing its online learning capability with the baseline method (ICO) and assessing its overall performance against state-of-the-art methods. To evaluate online learning, the drone was tasked with autonomously performing missions in two complex simulated environments: a high-density pole map and a forest map. The proposed method (ICP) outperformed the baseline (ICO), achieving success rates of 100% versus 80% in the pole map and 100% versus 40% in the forest map. To assess overall performance, two comparative analyses were conducted. First, in terms of computational efficiency, our method required 42.15 ms on the same testing machine, significantly outperforming [9] at 259.48 ms, though it remained higher than [7] at 17.59 ms. A deeper analysis was conducted by separating computation time into two components: perception processing and control processing. Interestingly, perception processing accounted for 99% of our total computation time, taking 41.72 ms, which was higher than both [9] (13.28 ms) and [7] (7.64 ms). In contrast, our control system processing was highly efficient, requiring only 0.43 ms, outperforming both [9] (13.28 ms) and [7] (9.95 ms). Second, we evaluated the system based on the overall mission completion time and success rate in three different environments: large obstacles (long walls), multiple small obstacles (poles), and dynamic small obstacle (suddenly appearing pole). The proposed method successfully completed missions in all environments with a 100% success rate, whereas both [9] and [7] failed in the long-walled environment but

succeeded in the multiple and dynamic-obstacle scenarios. In terms of overall mission completion time, our method outperformed [9], but was still slower than [7]. Our method focuses on adaptive obstacle avoidance without considering goal direction in its turning decisions. While effective for avoiding obstacles, this can lead to less efficient paths and extended completion times. Conversely, the method in [7] includes goal direction, enabling more direct navigation. To demonstrate the performance and robustness of our proposed method, we deployed our system to successfully navigate the real drone in a real-world environment, handling both static and dynamic obstacles under both daytime and nighttime conditions.

In the future, we plan to enhance our system by focusing on optimizing the perception processing system to reduce computation time. Additionally, integrating goal-directed planning into the decision-making process will enable more efficient turning, allowing the drone to avoid obstacles while moving toward the goal, ultimately completing missions in less time.

REFERENCES

- [1] U. R. Mogili and B. B. V. L. Deepak, "Review on application of drone systems in precision agriculture," *Procedia Computer Science*, vol. 133, pp. 502–509, 2018.
- [2] X. Liu, G. V. Nardari, F. C. Ojeda, Y. Tao, A. Zhou, T. Donnelly, C. Qu, S. W. Chen, R. A. F. Romero, C. J. Taylor, and V. Kumar, "Large-scale autonomous flight with real-time semantic slam under dense forest canopy," *IEEE RA-L*, vol. 7, pp. 5512–5519, APR 2022.
- [3] M. Kulkarni, M. Dharmadhikari, M. Tranzatto, S. Zimmermann, V. Reijgwart, P. De Petris, H. Nguyen, N. Khedekar, C. Papachristos, L. Ott, R. Siegwart, M. Hutter, and K. Alexis, "Autonomous teamed exploration of subterranean environments using legged and aerial robots," in *ICRA2022*, pp. 3306–3313.
- [4] B. Lindqvist, A.-A. Agha-Mohammadi, and G. Nikolakopoulos, "Exploration-rtt: A multi-objective path planning and exploration framework for unknown and unstructured environments," in *IROS2021*, pp. 3429–3435.
- [5] Z. Xu, D. Deng, and K. Shimada, "Autonomous uav exploration of dynamic environments via incremental sampling and probabilistic roadmap," *IEEE RA-L*, vol. 6, no. 2, pp. 2729–2736, 2021.
- [6] X. Zhou, Z. Wang, H. Ye, C. Xu, and F. Gao, "Ego-planner: An esdf-free gradient-based local planner for quadrotors," *IEEE RA-L*, vol. 6, no. 2, pp. 478–485, 2021.
- [7] J. Zhang, C. Hu, R. G. Chadha, and S. Singh, "Falco: Fast likelihood-based collision avoidance with extension to human-guided navigation," *Journal of Field Robotics*, vol. 37, no. 8, pp. 1300–1313.
- [8] Y. Song and D. Scaramuzza, "Policy search for model predictive control with application to agile drone flight," *IEEE T-RO*, vol. 38, no. 4, pp. 2114–2130, 2022.
- [9] R. Szczepanski, T. Tarczewski, and K. Erwinski, "Energy efficient local path planning algorithm based on predictive artificial potential field," *IEEE Access*, vol. 10, pp. 39 729–39 742, 2022.
- [10] A. Loquercio, E. Kaufmann, R. Ranftl, M. Müller, V. Koltun, and D. Scaramuzza, "Learning high-speed flight in the wild," *Science Robotics*, vol. 6, p. Not available, OCT 2021.
- [11] G. Zhao, T. Wu, Y. Chen, and F. Gao, "Learning speed adaptation for flight in clutter," *IEEE RA-L*, vol. 9, pp. 7222–7229, AUG 2024.
- [12] Y. Cai, W. Xu, and F. Zhang, "ikd-tree: An incremental kd tree for robotic applications," *arXiv preprint arXiv:2102.10808*, 2021.
- [13] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [14] V. Jaiton, K. Rothomphiwat, E. Ebeid, and P. Manoonpong, "Neural control and online learning for speed adaptation of unmanned aerial vehicles," *Frontiers in Neural Circuits*, vol. 16, 2022.
- [15] C. K. Pedersen and P. Manoonpong, "Neural control and synaptic plasticity for adaptive obstacle avoidance of autonomous drones," in *From Animals to Animats 15*. Springer, 2018, pp. 177–188.